

PvrGPU PipelineTxn — 資料結構與管線交易機制詳解

本文件詳述 PvrGPU SystemC 模型中核心交易控制結構 **PipelineTxn** (Pipeline Transaction) 的定義、設計哲學、關聯資料結構 (**PoolHandle**、**PipelineState**、**MemoryTxn**) 以及在全 GPU 管線中的生命週期與資料流向。

0. 架構總覽與設計哲學

SystemC Bounded FIFO Communication Architecture

管線模組間透過 `sc_fifo<PipelineTxn>` 傳遞輕量級控制 Token
巨量資料 (頂點、三角形、紋理、像素) 均留存在 MemoryPool

↓ [Module A (e.g. VDM)] [Module B (e.g. VertexFetch)]

讀取 `txn = fifo.read` | | 讀取 `txn = fifo.read`

修改 PipelineState | | 讀取/修改 PipelineState

寫出 `fifo.write(txn)` | | 寫出 `fifo.write(txn)`

↓ `sc_fifo<PipelineTxn>` `sc_fifo<PipelineTxn>` (僅傳遞 24-byte Token) (僅傳遞 24-byte Token)

↓ MemoryPool | +-----+ | PipelineState (完整 Draw Call 狀態控制塊): | | - stage
(管線階段守衛, e.g. kSubmitted -> kVdmComplete) | | - functional_case, width, height, counters | | - 子陣列 Handle: vertex_lanes,
raster_triangles, tile_records, | | parameter_triangles, fragment_quads, pbe_framebuffer | |
+-----+

設計核心原則：

1. Zero-Copy Handle 機制：GPU 管線傳輸的幾何與像素數據極大，SystemC `sc_fifo` 若頻繁深拷貝 (Deep Copy) 物件將導致嚴重的模擬效能瓶頸。因此 FIFO 僅傳送含 **PoolHandle** 的輕量 Token。
2. 世代安全檢查 (Generation Check)： **PoolHandle** 帶有 **generation** 計數，防止 MemoryPool 槽位釋放或重用時出現懸置指標或 Use-After-Free 錯誤。
3. 單調性與因果順序 (Monotonic Ordering)：透過 **sequence** 與 **frame** 確保跨階段、跨非同步請求 (如 TPU 紋理採樣、SLC/DRAM 快取讀寫) 時維持嚴格的 API 提交先後順序。

1. 資料結構定義

定義於 `[model_stub/model_types.h]` (file:///Users/linwany/Library/Mobile%20Documents/com~apple~CloudDocs/Codex/Pvr GPU/model_stub/model_types.h#L186-L240)：

1.1 `PoolHandle` 結構

```
struct PoolHandle {
    std::uint32_t slot = 0;    // MemoryPool 中的槽位索引 (Slot Index)
    std::uint32_t generation = 0; // 世代計數器 (Generation Counter)
};
```

1.2 `PipelineTxn` 結構

```
struct PipelineTxn {
    PoolHandle state;    // 指向 MemoryPool 中 PipelineState 的控制代碼
    std::uint32_t frame = 0;    // 當前渲染影格編號 (Frame Number)
    std::uint64_t sequence = 0; // 單調遞增的指令序列編號 (Sequence ID)
};
```

1.3 欄位詳細說明

欄位名稱	型別	位元組大小	說明與用途
state	PoolHandle	8 Bytes	指向共用 MemoryPool 中的 **PipelineState** 。每個 SystemC 模組在 Run() 迴圈起始時呼叫 LoadPipelineState(pool_, txn.state) 取得狀態，處理完成後呼叫 StorePipelineState(pool_, txn.state, state) 並將 txn 寫入下游 FIFO。
frame	uint32_t	4 Bytes	代表當前 Draw Call / Clear 屬於第幾影格（用於多影格追蹤、Profile 記錄與除錯）。
sequence	uint64_t	8 Bytes	Draw Call 提交的唯一全域單調遞增序號，確保管線中多個交錯的 Transaction 保持嚴格順序。

總計結構大小：20 Bytes（64-bit 記憶體對齊後為 24 Bytes）。

2. 記憶體交易衍生結構 — `MemoryTxn`

當管線模組需要與記憶體階層（TCU 紋理快取、MCU 混合快取、USC L2 快取、SLC 或 DRAM）互動時，**PipelineTxn** 會被包裝在 **MemoryTxn** 內傳輸：

```
enum class MemoryOperation : std::uint8_t {
    kRead = 0,
    kWrite = 1,
};

enum class MemoryClient : std::uint8_t {
    kFramebuffer = 0,
    kMixedCache = 1,
    kTextureCache = 2,
    kUscl2 = 3,
    kTextureUpload = 4,
    kIndexFetch = 5,
    kVertexFetch = 6,
    kParameterWrite = 7,
    kParameterRead = 8,
    kFramebufferReadback = 9,
};

enum class MemoryPayloadFormat : std::uint8_t {
    kLinearBytes = 0,
    kCacheLineWrites = 1,
};

struct MemoryTxn {
    PipelineTxn    pipeline;    // 原始管線交易 (保留 state, frame, sequence)
    PoolHandle     payload;     // 資料本體在 MemoryPool 中的 Handle (寫入或回傳資料)
    std::uint64_t  address = 0; // GPU 實體或虛擬位址 (GPU Address)
    std::uint64_t  bytes = 0;   // 存取長度 (Byte Count)
    std::uint64_t  request_id = 0; // 請求序號 (用於快取/記憶體回應匹配)
    MemoryOperation operation;   // kRead 或 kWrite
    MemoryClient   client;      // 發起客戶端類型 (e.g. kTextureCache, kFramebuffer)
    MemoryPayloadFormat payload_format; // 資料格式 (線性位元組或快取行)
    std::uint8_t   reserved[5]; // 保留欄位
};
```

3. `PipelineTxn` 於全管線模組之流動與生命週期

在典型渲染中，單一 **PipelineTxn** 從建立到銷毀經歷如下生命週期：

① Submitter

建立初始 PipelineState，存入 MemoryPool 取得 PoolHandle
產生 PipelineTxn { state: h0, frame: 0, sequence: 1 }
STAGE: kSubmitted

↓ sc_fifo<PipelineTxn>

② Geometry Phase

VDM -> VertexFetch -> VertexPDS -> PcoDecoder -> UscSlot -> UscCluster
-> ClipCull -> Tiler -> ParameterBuffer
* 每個模組讀取同一 txn，更新 state.stage 與幾何資料 Handle
STAGE: kVdmComplete -> ... -> kParameterBufferReady

↓ sc_fifo<PipelineTxn>

③ Fragment & Texture Phase

FragmentDecoder -> TileScheduler -> ISP -> FragmentFrontend -> PDS
-> UscSlot -> UscCluster <-> TextureUnit (SMP Request/Response) -> PBE
* ISP 產生 fragment_candidates，USC 執行著色，PBE 合成 RGBA8 Framebuffer
STAGE: kTilesScheduled -> ... -> kPbeComplete

↓ sc_fifo<PipelineTxn>

④ Write-Back & Complete Phase

PbeWriteBack: 將 Framebuffer 寫入 DRAM (0x10000000) 並 Readback 驗證
JsonReporter: 驗證 MemoryPool 分配與釋放平衡，產生 JSONL 報告
STAGE: kPixelDataMasterComplete -> kFramebufferReady

4. 關鍵輔助函式與安全性驗證

4.1 載入與儲存 PipelineState

```
// 透過 txn.state 從 MemoryPool 載入 PipelineState 控制塊  
PipelineState state = LoadPipelineState(pool_, txn.state);
```

```
// 驗證管線階段守衛 (Stage Guard)，不符合立即 Fail-Closed 擲出異常  
RequireStage(state.stage, PipelineStage::kSubmitted, "VDM");
```

```
// 模組運算完成後，更新 stage 並存回 MemoryPool  
state.stage = PipelineStage::kVdmComplete;  
StorePipelineState(pool_, txn.state, state);
```

```
// 將相同的 txn 傳送至下游模組  
output_fifo.write(txn);
```

4.2 世代安全性 (Generation Safety)

MemoryPool 內部為每個 Slot 維護 **generation**：

- 當物件分配時，**generation** 遞增。
- 當讀取 **PoolHandle** 時，若 **handle.generation != slot.generation**，視為非法參照，防止讀取已釋放或被新物件覆寫的空間。
- 模擬結束時，**JsonReporter** 檢查 **pool.allocations() == pool.releases()** 且 **pool.bytes_in_flight() == 0**，確保零記憶體洩漏。

5. 總結

特性	說明
本質	20 位元組的輕量 SystemC 交易控制權杖 (Token)
核心成員	PoolHandle state (MemoryPool 指標)、 frame (影格)、 sequence (指令序號)
效能優勢	避免在 SystemC FIFO 複製龐大幾何/像素資料，實現零拷貝傳遞
衍生交易	包裝於 MemoryTxn 中，提供統一的快取與 DRAM 存取介面
安全機制	世代計數器防懸置指標，階段守衛防亂序執行，結束時保證記憶體回收